INFRASTRUCTURE AS CODE · CERTIFICATION PREPARATION

Chapter 01

Introduction to Infrastructure as Code & first deployment with Terraform

Understand the principles of IaC, discover Terraform and prepare the working environment to carry out a first AWS deployment.

Terraform | AWS | HCL | Providers | State | Workflow

KEY IDEA

With Terraform, you describe the **desired state** of an infrastructure in configuration files. Terraform compares this configuration with its known state and communicates with the **providers** to create, modify or delete the necessary resources.

Contents

1. Why Infrastructure as Code? . . . 2
2. Introduction to Terraform . . . 3
3. Installation and environment setup . . . 6
4. First AWS deployment with Terraform . . . 7
5. What you absolutely must remember . . . 8
6. Comprehension questions . . . 8

1 Why Infrastructure as Code?

1.1 Traditional infrastructure management

In a traditional approach, an administrator configures the infrastructure through graphical interfaces or manually executed commands. For example, to deploy a **web server**, they may have to:

1. Log in to the cloud provider’s console.
2. Create a virtual instance.
3. Choose a system image and an instance size.
4. Configure the network and security rules.
5. Start the instance.
6. Install and configure the web server.

LIMITATIONS OF THE MANUAL APPROACH

- Manual operations are **repetitive**.
- A configuration error can be **hard to detect**.
- It is hard to **reproduce exactly** the same environment.
- Documentation may not reflect the infrastructure actually deployed.
- Changes are hard to **track and audit**.

1.2 Definition of Infrastructure as Code

Infrastructure as Code (IaC) is an approach that consists of defining and managing the components of an infrastructure using configuration files or code. Instead of creating resources manually, the administrator **describes the desired state**; an IaC tool interprets this description and carries out the necessary operations.

Examples of resources that can be managed with IaC

Virtual machines	**Networks and subnets**	**Security rules**
Load balancers	**Databases**	**Storage buckets**

…as well as users and their permissions.

1.3 Manual approach vs Infrastructure as Code

Manual approach	Infrastructure as Code
Creation via a console or one-off commands	Description in configuration files
Often repetitive operations	Automatable operations
Reproduction sometimes difficult	Reusable configuration
Changes hard to trace	History possible with Git
Risk of inconsistencies between environments	Possibility of standardizing environments

1.4 The main benefits of IaC

Automation
Reduce manual operations.

Repeatability
Reproduce an infrastructure from a configuration.

Versioning
Track changes with Git.

Consistency
Apply a common configuration to several environments.

Traceability
Review the proposed changes before applying them.

Collaboration
Share and review configurations between team members.

1.5 Imperative and declarative approaches

Imperative approach

The user describes the **actions to run** and their **order**.

Conceptual example

1. Create a network.
2. Create a virtual machine.
3. Attach the machine to the network.
4. Add a security rule.

Declarative approach

The user describes the **desired result**; the tool determines the necessary operations.

Conceptual example

I want:

- a network;
- a virtual machine connected to this network;
- a rule allowing HTTP traffic.

TO REMEMBER
Terraform mainly uses a **declarative** approach.

2 Introduction to Terraform

2.1 What is Terraform?

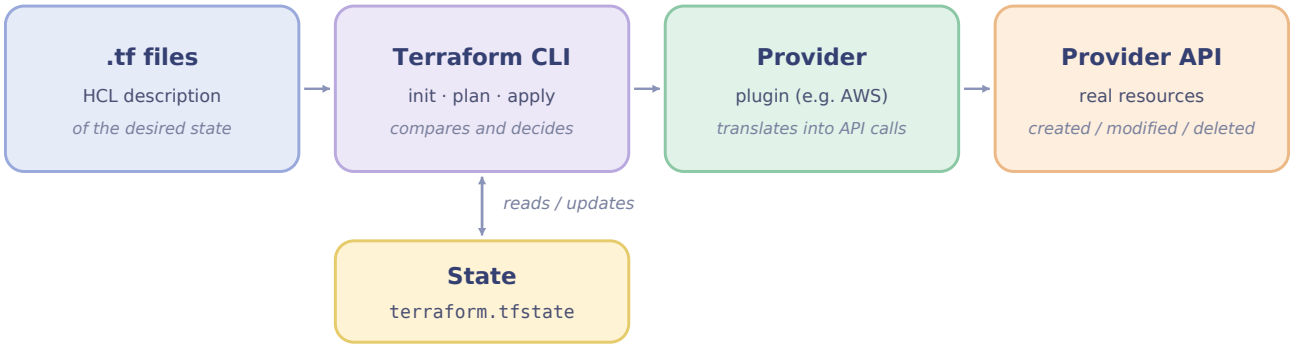
Terraform is an Infrastructure as Code tool developed by **HashiCorp**. It lets you define, provision and manage infrastructure resources using configuration files. It communicates with various vendors and platforms through **providers**.

Examples of providers

AWS	**Microsoft Azure**	**Google Cloud Platform**
VMware vSphere	**Docker**	**Kubernetes**

2.2 How Terraform works

The user describes the infrastructure in **.tf** files. Terraform reads these files, consults its state and communicates with the providers' APIs to determine and carry out the necessary changes.



2.3 The essential components

Terraform CLI	Command-line tool used to **initialize** a project, **check** the configuration, **prepare a plan** and **apply** the changes.
Provider	Plugin that lets Terraform interact with a platform or a service. *Example: the AWS provider lets you manage AWS resources.*
Resource	Component that Terraform can create, modify or delete. *Examples:* `aws_instance` `aws_security_group` `aws_s3_bucket`
Data Source	Lets you **read** information about an existing resource or object, without necessarily creating it.
Terraform State	Data file that records the resources managed by Terraform and maps the configuration to the real objects.
HCL Configuration	Terraform uses the **HashiCorp Configuration Language** (HCL) to describe the infrastructure.

```hcl
resource "aws_instance" "web" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t3.micro"
}
```

This snippet declares an **EC2 resource** managed by the AWS provider.

2.4 The Terraform workflow

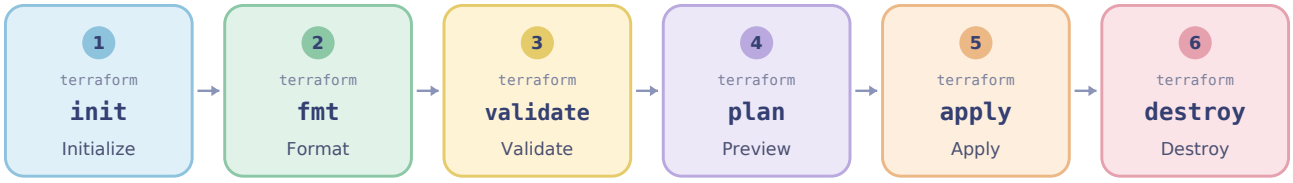
The basic workflow consists of several commands, to be run in the configuration directory.

2.4.1	`terraform init`	Initializes the working directory. Downloads in particular the required **providers** and prepares the **.terraform** directory.
2.4.2	`terraform fmt`	Formats the Terraform files according to Terraform conventions.
2.4.3	`terraform validate`	Checks the **syntactic and structural** consistency of the configuration.
2.4.4	`terraform plan`	Shows the changes Terraform plans to make. It compares the configuration with the information in the State and the remote infrastructure.
2.4.5	`terraform apply`	Applies the changes needed to reach the desired state. A plan is displayed and a **confirmation** is requested, unless options say otherwise.
2.4.6	`terraform destroy`	Deletes the resources managed by the relevant configuration and State.

GOOD TO KNOW — TERRAFORM VALIDATE
This command **does not guarantee** that the resources can actually be created at the cloud provider.

WARNING — TERRAFORM DESTROY
This command can cause **real deletion** of cloud resources. Always check the plan and the targeted environment before confirming.

2.4.7 Workflow overview



Command summary

Command	Role
`terraform init`	Initialize the project
`terraform fmt`	Format the configuration
`terraform validate`	Check the configuration
`terraform plan`	Preview the changes
`terraform apply`	Apply the changes
`terraform destroy`	Destroy the managed resources

3 Installation and environment setup

3.1 Installing Terraform on Windows

Consult the official documentation (*Install Terraform*) to choose the installation method that suits you. After installation, check the version:

```
POWERSHELL
./terraform.exe --version
```

Then add the Terraform folder to the Windows **Path** environment variable:

1. Press **Windows + R**, then type `sysdm.cpl`.
2. Go to the **Advanced** tab › **Environment Variables**.
3. Under **User variables**, select **Path** › **Edit**.
4. Click **New** and paste the folder path, for example:

```
PATH
C:\Users\Admin\Downloads\terraform_1.17.1_windows_amd64
```

3.2 Setting up Visual Studio Code

Install **Visual Studio Code**, then the **HashiCorp Terraform** extension to get, among other things:

Coloring	**Auto-completion**	**Formatting**	**Error detection**
HCL syntax.	Suggestions for blocks and arguments.	File formatting.	Some syntax errors.

3.3 Configuring AWS credentials

Terraform must be able to authenticate with AWS to create and manage resources. The method used in this lab is configuring an **AWS profile** with the **AWS CLI** (to be installed if necessary).

Configure a profile

```
TERMINAL
aws configure --profile terraform-lab
```

The AWS CLI asks in particular for the following information (in the Learner Lab: **AWS Details** › **Show**):

AWS Access Key ID	**AWS Secret Access Key**	**Default region name** Example: us-east-1	**Default output format**

Verify the profile

```
TERMINAL
aws configure list --profile terraform-lab
```

Verify the identity associated with the profile:

```
TERMINAL
aws sts get-caller-identity --profile terraform-lab
```

3.4 Configuring the provider

Revisit the AWS provider documentation, then create a file named **provider.tf** containing:

HCL · PROVIDER.TF

```hcl
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.72.0"
    }
  }
}

provider "aws" {
  # Configuration options
  region  = "us-east-1"
  profile = "terraform-lab"
}
```

Then, in the directory containing `provider.tf`, run:

TERMINAL

```
terraform init
```

PATH ERROR?
If the **terraform** command is not recognized, add the folder to the Path of the current session:

POWERSHELL

```
$env:Path += ";C:\Users\Admin\Downloads\terraform_1.17.1_windows_amd64"
```

4 First AWS deployment with Terraform

PRACTICAL WORK
The first deployment is the subject of a dedicated lab. *(Link to the lab to be added here.)*

5 What you absolutely must remember

Infrastructure as Code	IaC consists of describing and managing an infrastructure using configuration files or code.
Terraform	IaC tool that lets you provision and manage resources through various providers.
Provider	Plugin that lets Terraform communicate with a platform or a service.
Resource	Infrastructure component that Terraform can manage.
Terraform Workflow	`init` prepares the project, `plan` previews the changes, `apply` applies them and `destroy` deletes the managed resources.
Terraform State	Lets Terraform track resources and link the configuration to the real infrastructure.
HCL	Configuration language used to describe the resources and Terraform parameters.

6 Comprehension questions

Q1	What is Infrastructure as Code? Name three advantages over a manual configuration.
Q2	What is the difference between an imperative approach and a declarative approach?
Q3	What is the role of Terraform?
Q4	What is a Terraform provider? Give two examples.
Q5	What is the difference between a resource and a data source?
Q6	What is the role of the `terraform init` command?
Q7	What is the difference between `terraform plan` and `terraform apply` ?
Q8	Does the `terraform validate` command guarantee that the resources can be created on AWS? Justify.
Q9	What is the role of the `terraform.tfstate` file?
Q10	Why should the State file not be published in a public GitHub repository?

Q11

In the following block, identify the **resource type**, its **logical name** and the **provider** used:

```hcl
resource "aws_instance" "web" {
  instance_type = "t3.micro"
  ami           = "ami-xxxxxxxx"
}
```

Q12

Why is it important to run `terraform plan` before `terraform apply` ?